PROJECT REPORT | CLOUD SECURITY

AWS IAM Security and Least-Privilege Access Project

Role-based access control for Amazon S3 and EC2

Platform	AWS Identity and Access Management (IAM)
Related services	Amazon S3, Amazon EC2 (IAM roles)
Report date	25 October 2026
Prepared by	_____

Table of Contents

Table of Contents 2

1. Executive Summary 3

2. Objectives and Scope 3

3. Solution Architecture 4

4. AWS Services and Resources 5

5. Implementation Steps 6

6. Security Concepts Demonstrated 11

7. Validation Status 12

8. Findings and Recommendations 13

9. Limitations and Future Improvements 13

10. Cost Management 13

11. Conclusion 14

Appendix A – Developer Policy (intended final version) 15

Appendix B – Glossary 16

1. Executive Summary

This project demonstrates **AWS Identity and Access Management (IAM)** security practices through hands-on configuration of IAM user groups, users, a custom permissions policy for Amazon S3, and an IAM service role for Amazon EC2. The aim is to implement **role-based access control (RBAC)** and apply the **principle of least privilege** to AWS resource access.

Three working groups were created (**Developers**, **Auditors** and an administrators group). The Developers group received the custom policy **Developer-Project-S3-Access**, which allows listing the project bucket and getting, putting and deleting objects in it, and deliberately omits *s3:ListAllMyBuckets*. Auditors receive the AWS-managed **ReadOnlyAccess** policy. For services, the role **EC2-S3-Project-Role** with the policy **EC2-Project-S3-Access** lets EC2 reach the bucket with temporary credentials instead of stored keys.

The console screenshots confirm the groups, the developer policy attachment, the EC2 role and the user list. Live permission tests, EC2 instance attachment, MFA and CloudTrail are **not yet verified** and are listed as future work, as is an additional broadly privileged **Admin-Role** that this report recommends reviewing.

Item	Status
IAM groups	Developers, Auditors and an administrators group created; each shows 1 user and permissions Defined
Developer policy	Custom policy Developer-Project-S3-Access attached to Developers (customer managed)
Auditor policy	AWS-managed ReadOnlyAccess (per project notes; policy list not captured in a screenshot)
EC2 role	EC2-S3-Project-Role with EC2-Project-S3-Access created; last activity: none
Live access tests	Not performed / not recorded yet
MFA, CloudTrail, cross-account	Not implemented (future work)

2. Objectives and Scope

2.1 Objectives

- Understand IAM users, groups, roles and policies.
- Implement least-privilege access to Amazon S3.
- Separate developer and auditor responsibilities.
- Create a custom IAM permissions policy.
- Configure an EC2 IAM role for S3 access.
- Understand MFA, cross-account access and CloudTrail auditing.

2.2 In scope

- IAM user groups, users and group-level policy attachment.
- A custom S3 policy scoped to one project bucket (bucket and object ARNs).
- An EC2 service role and its S3 policy.

2.3 Out of scope (future work)

- MFA enforcement and privileged-access hardening; cross-account roles.

- CloudTrail audit logging; automated policy validation.
- Infrastructure as Code (Terraform) and automated compliance reporting.
- Live EC2 role attachment and access validation.

3. Solution Architecture

Permissions are never given directly to individual people. Users inherit access from groups, groups carry policies, and AWS services use a role. This keeps each identity limited to the actions and resources its job requires.

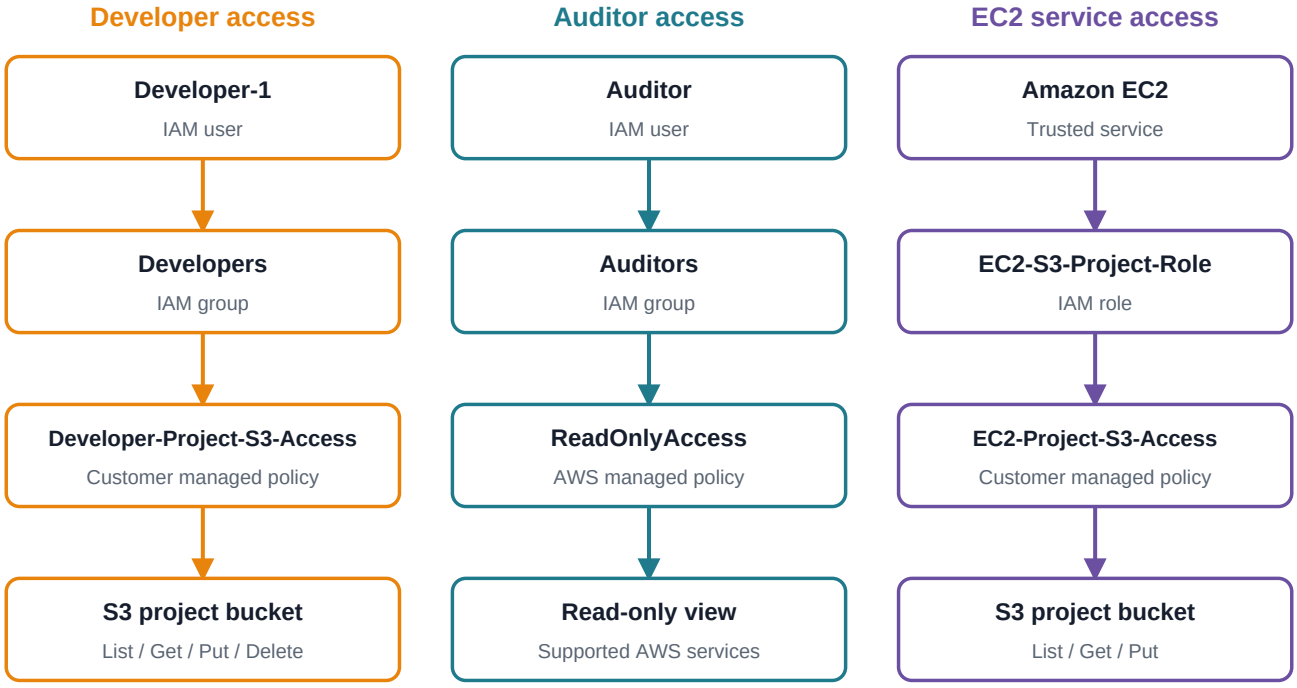


Figure 1 – Access architecture: developer, auditor and EC2 service paths.

4. AWS Services and Resources

Resource	Name	Type / notes
Service	AWS Identity and Access Management	Identity, groups, policies, roles
Service	Amazon S3	Dedicated project bucket (iam-security-project-...)
Service	Amazon EC2 (IAM role only)	Role trusted by EC2; no running instance confirmed
IAM group	Developers	Custom S3 policy attached
IAM group	Auditors	ReadOnlyAccess (AWS managed)
IAM group	Administrators group (named “Adminstrators” in the console)	Four AWS-managed policies incl. AdministratorAccess
IAM users	Developer-1, Auditor, Admin, Parikshit	Each of the first three belongs to one group; Parikshit belongs to none
Custom policy	Developer-Project-S3-Access	Customer managed; ListBucket, Get/Put/DeleteObject
Custom policy	EC2-Project-S3-Access	Customer managed; list, read, upload
IAM role	EC2-S3-Project-Role	Trusted entity: EC2; max session 1 hour
IAM role	Admin-Role	Additional EC2 role; 3 AWS-managed policies (see Section 8)

Note: the project notes call the users `developer-user` and `auditor-user`; the console screenshots show **Developer-1** and **Auditor**. This report uses the names visible in the console.

5. Implementation Steps

Step 1 – Create IAM groups

Groups were created so that permissions are managed by responsibility. Immediately after creation the three groups had **0 users** and permissions **Not defined** (Figure 2). After users were added and policies attached, each group showed **1 user** and permissions **Defined** (Figure 3).

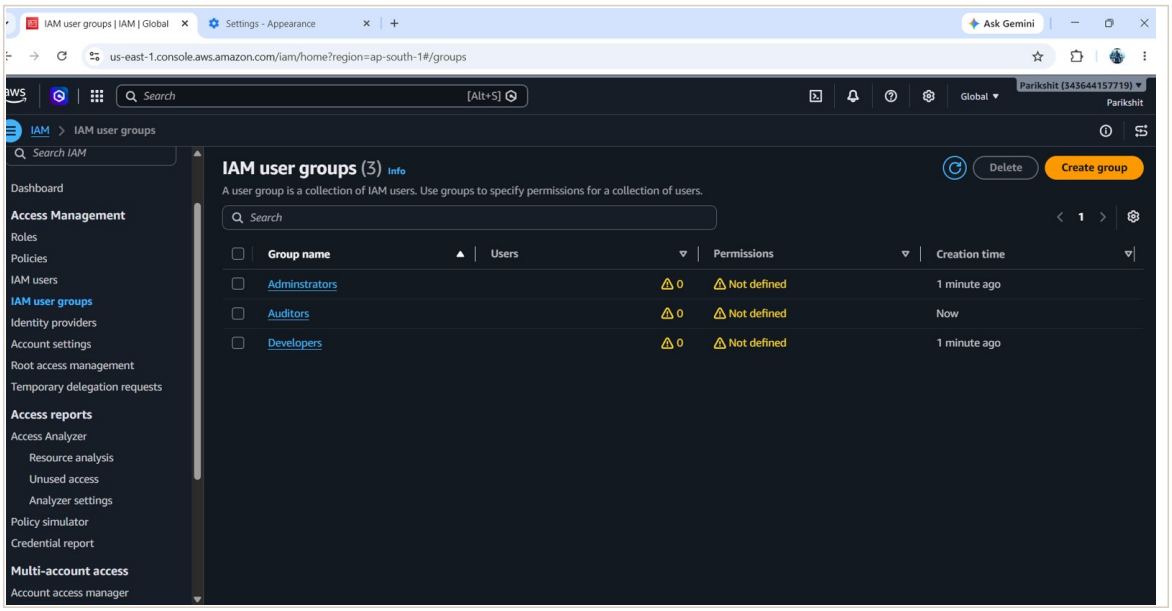


Figure 2 – IAM user groups right after creation: 0 users, permissions not defined.

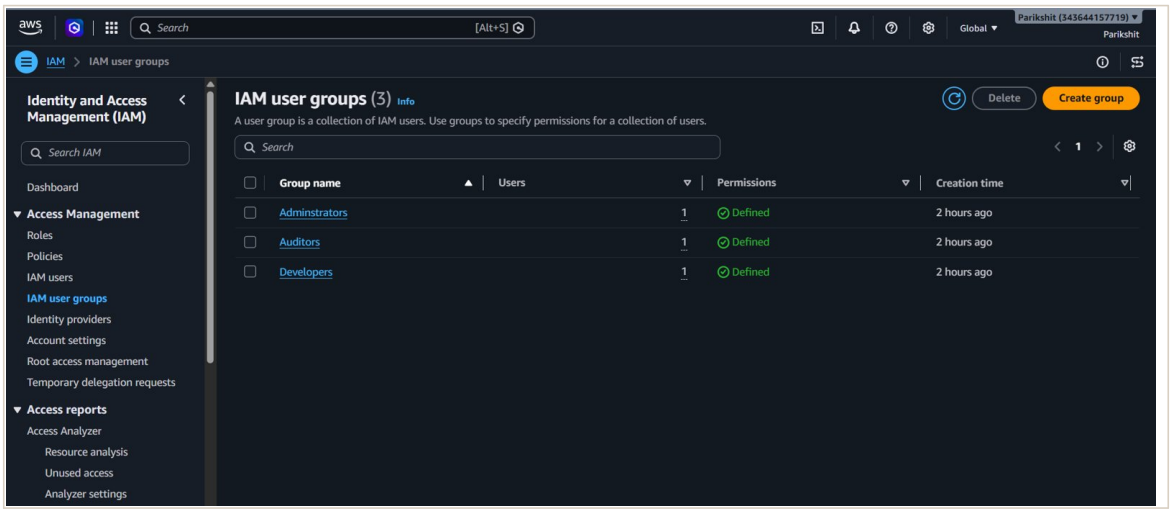


Figure 3 – IAM user groups after configuration: 1 user each, permissions defined.

Step 2 – Developer access with a custom policy

The custom policy **Developer-Project-S3-Access** was written in the IAM JSON editor. It scopes access to the project bucket using ARNs: `s3:ListBucket` on the bucket, and `s3:GetObject`, `s3:PutObject` and `s3:DeleteObject` on `bucket/*`. The editor screenshot shows an initial first statement *ListBuckets* granting `s3:ListAllMyBuckets` on `*` selected for editing; this statement was then removed so developers cannot enumerate every bucket name in the account.

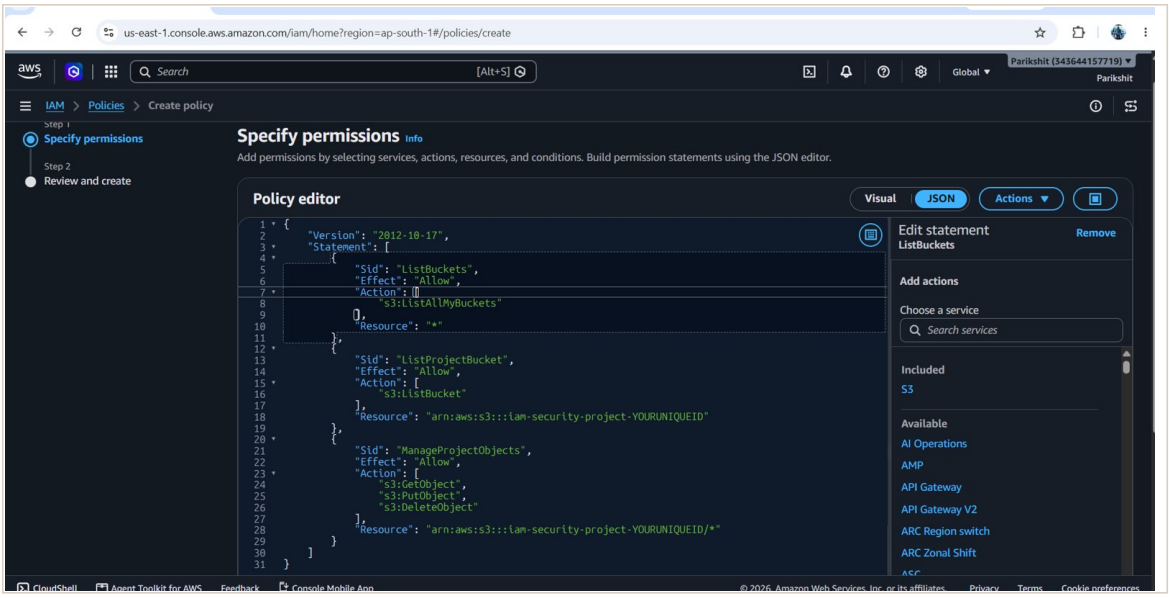


Figure 4 – Policy editor (JSON): ListBuckets statement being edited, project bucket and object statements.

The policy was attached to the Developers group; the group's Permissions tab lists it as a **customer managed** policy with 1 attached entity.

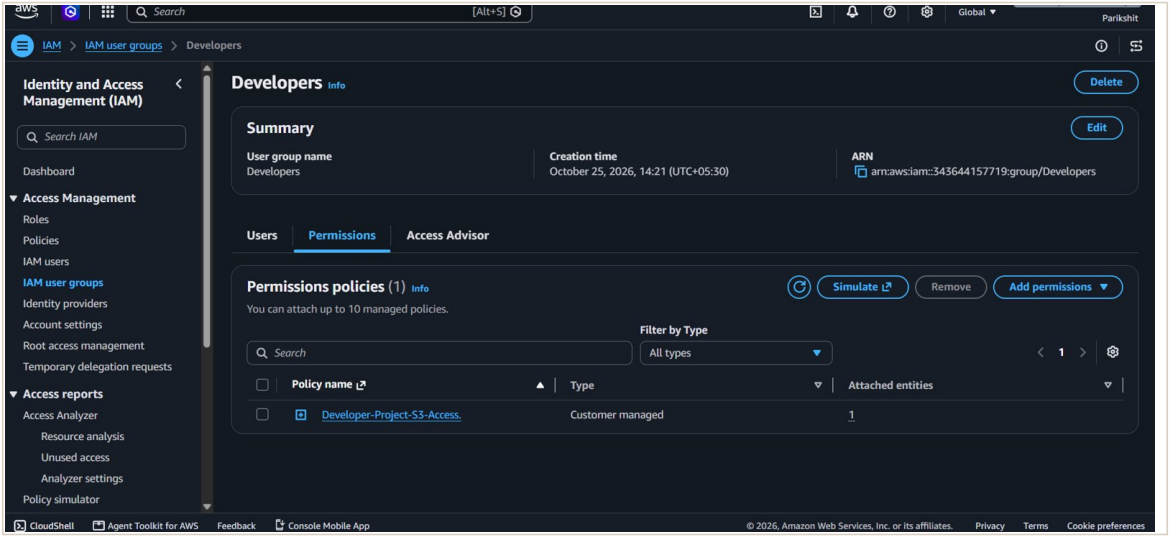


Figure 5 – Developers group: Developer-Project-S3-Access attached (created 25 Oct 2026, 14:21 UTC+05:30).

Effect of removing ListAllMyBuckets: the developer can no longer browse the bucket list in the S3 console home page, but can open the project bucket directly (console URL, CLI or SDK) because ListBucket on that bucket is still allowed. This is the intended least-privilege behaviour.

Step 3 – Auditor access

The user **Auditor** was placed in the **Auditors** group, which carries the AWS-managed **ReadOnlyAccess** policy. This gives view-only access across supported AWS services and no ability to change resources, separating auditing from development. ReadOnlyAccess is broad; where auditors only need selected services, a narrower custom policy is preferable in production. The user list in Figure 6 shows the three role-based users, each in one group.

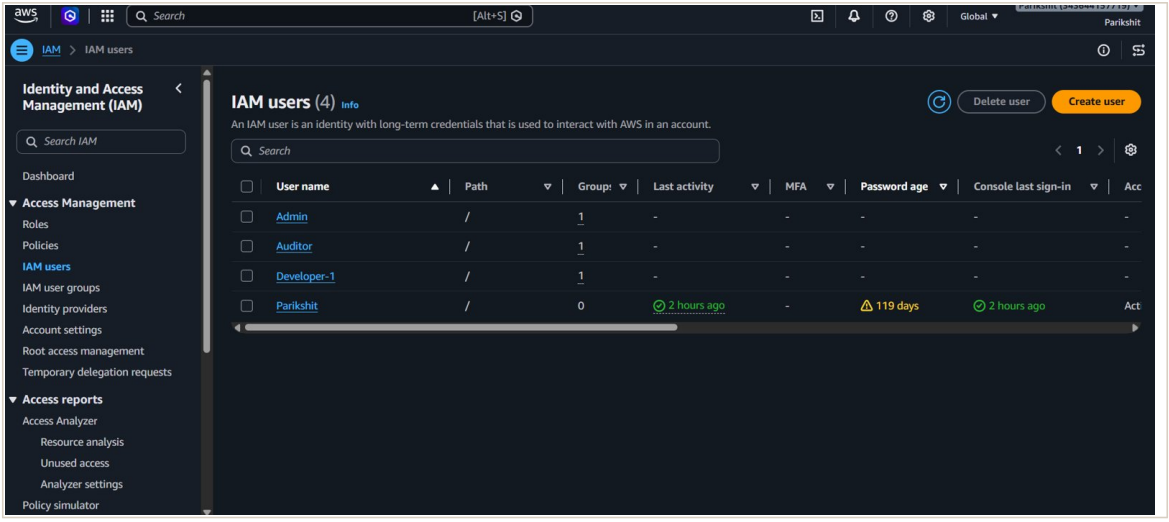


Figure 6 – IAM users: Admin, Auditor and Developer-1 (1 group each); Parikshit (0 groups). MFA column is empty for all users.

Step 4 – Administrators group

A third group, shown in the console as **Adminstrators** (spelling as created), holds four AWS-managed policies: AccountManagementFromVercel, AdministratorAccess, AdministratorAccess-Amplify and AdministratorAccess-AWSElasticBeanstalk. The user **Admin** belongs to it. The project notes name this group *CloudAdministrators*.

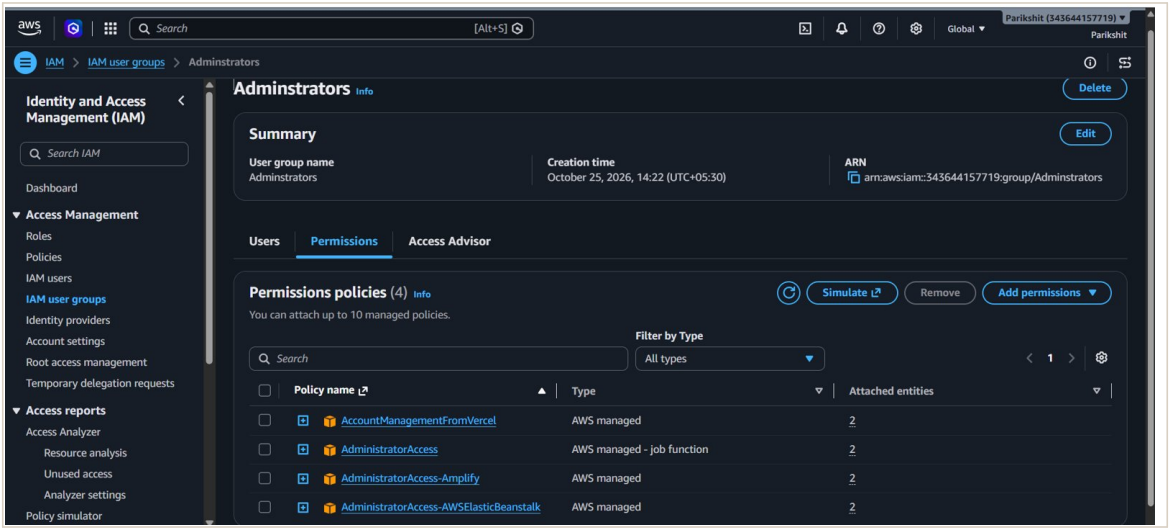


Figure 7 – Administrators group: four AWS-managed policies attached.

Step 5 – EC2 role for S3 access

The policy **EC2-Project-S3-Access** allows listing the project bucket, reading objects and uploading objects. The role **EC2-S3-Project-Role** (“Allows EC2 instances to call AWS services on your behalf”) trusts the EC2 service and carries this policy. An instance profile with the same name exists, which is what an EC2 instance uses to assume the role. *Last activity* is empty and the maximum session duration is 1 hour.

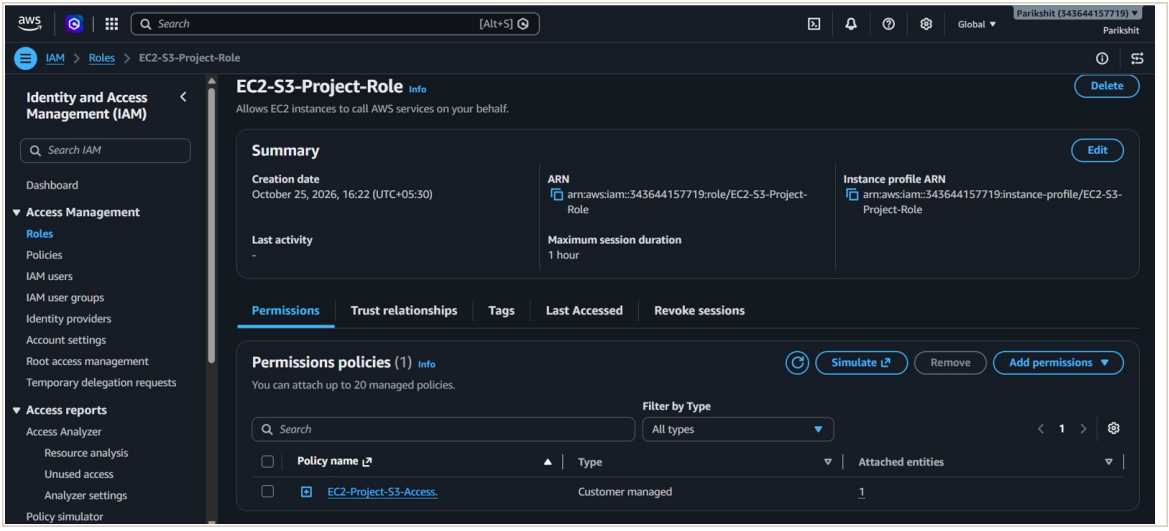


Figure 8 – EC2-S3-Project-Role with the customer managed policy EC2-Project-S3-Access (created 16:22).

Benefit: applications on EC2 receive short-lived temporary credentials automatically, so no access keys are stored on the instance. Attachment to a running instance and an access test were **not** performed.

Step 6 – Additional Admin-Role

A second role, **Admin-Role**, was also created (16:27) using the same EC2 trust relationship and description. It has three AWS-managed policies attached, starting with AccountManagementFromVercel, and the next row in the list appears to be AdministratorAccess. This role is outside the least-privilege design and is reviewed in Section 8.

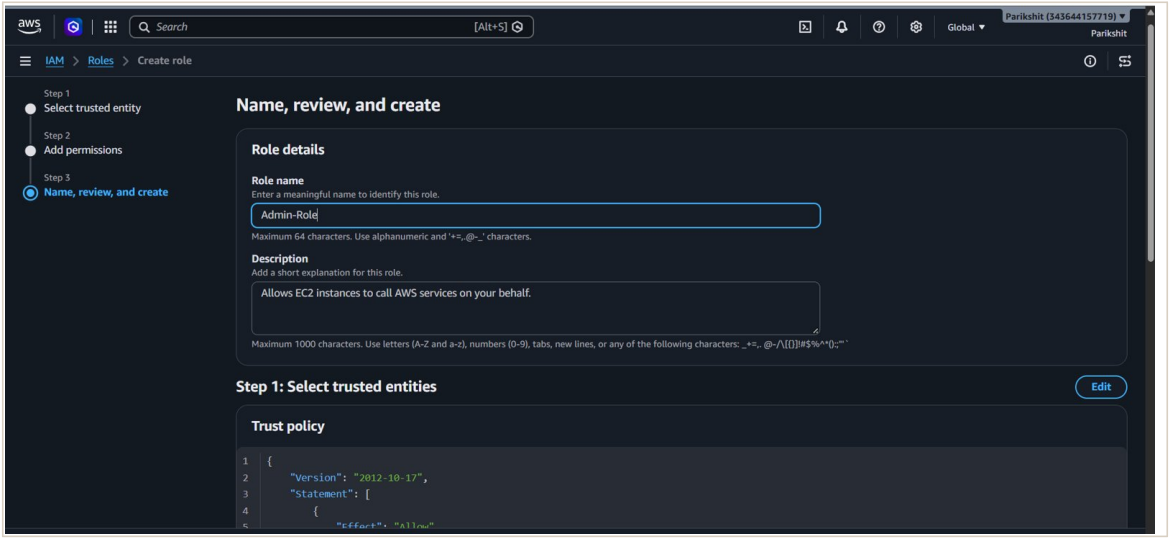


Figure 9 – Create role: naming Admin-Role with the EC2 trust description.

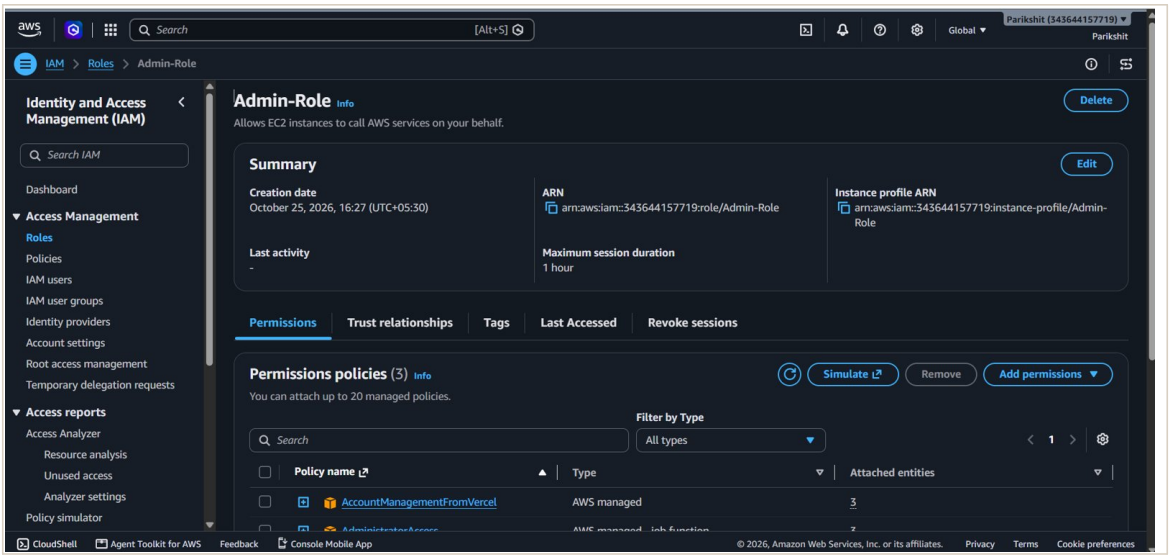


Figure 10 – Admin-Role summary: 3 permissions policies attached.

6. Security Concepts Demonstrated

Concept	How it is shown in this project
Least privilege	Developers get only ListBucket and object Get/Put/Delete on one bucket; no ListAllMyBuckets; the EC2 role has no delete permission.
Role-based access control	Permissions are attached to groups (Developers, Auditors), not to individual users.
Separation of duties	Developer (read/write/delete in one bucket) and auditor (read-only) access are held by different identities.
IAM roles	EC2-S3-Project-Role gives the EC2 service temporary credentials instead of long-term keys.
Resource-level permissions	S3 bucket ARN for ListBucket and bucket/* ARN for object actions.

7. Validation Status

Activity	Status	Evidence
Groups created (3)	Done	Figures 2, 3
Developer and auditor users created and grouped	Done	Figure 6
Custom developer policy attached to Developers	Done	Figure 5
ReadOnlyAccess attached to Auditors	Done per notes	Group shows permissions defined; policy list not captured
ListAllMyBuckets removed from developer policy	Done per notes	Figure 4 shows the editing step; final JSON not captured
EC2 role and S3 policy created	Done	Figure 8
Developer / auditor live permission tests	Not recorded	Test results to be added (allowed vs denied actions)
EC2 instance attachment and S3 access test	Not verified	Role shows no last activity
MFA, CloudTrail, cross-account roles	Not implemented	MFA column empty in Figure 6

Suggested test plan

- Sign in as **Developer-1**: list, upload, download and delete an object in the project bucket (expected: allowed); open S3 bucket list and another bucket (expected: denied).
- Sign in as **Auditor**: view IAM and S3 configuration (expected: allowed); upload or delete an object, change a policy (expected: denied).
- Launch a test EC2 instance with **EC2-S3-Project-Role**; run `aws s3 ls s3://<bucket>` and a test upload (expected: allowed); try delete (expected: denied). Terminate the instance afterwards.
- Use the IAM **Policy simulator** for each identity and record the results in a table.

8. Findings and Recommendations

#	Observation	Recommendation
1	**Admin-Role** is an EC2 role with three AWS-managed policies (the list appears to include AdministratorAccess). Any instance using it would have near-full account access.	Delete it if unused, or replace it with a scoped role. Never attach administrator permissions to an EC2 instance role.
2	No MFA is shown for any IAM user. User Parikshit has a password age of 119 days (warning) and belongs to no group.	Enforce MFA for all console users, rotate old credentials and give permissions through groups.
3	The administrators group name is misspelled (“Adminstrators”) and differs from the notes (CloudAdministrators). User names also differ from the notes.	Use consistent naming in the console and documentation; recreate the group if renaming is not possible.
4	The policy editor still shows the placeholder bucket name `iam-security-project-YOURUNIQUEID`.	Confirm the real bucket name is in the saved policy, otherwise the policy grants nothing useful.
5	ReadOnlyAccess for auditors covers many services and data-read actions.	Replace with SecurityAudit / ViewOnlyAccess or a custom read-only policy limited to the services being audited.
6	Developers can delete objects in the bucket.	Enable S3 versioning (and optionally MFA delete or Object Lock) so deletions are recoverable.
7	No audit trail or policy checking is in place.	Enable CloudTrail, review IAM Access Analyzer findings and run policy validation before attaching policies.
8	Screenshots show the AWS account ID in the header and ARNs.	Blur the account ID before publishing the report or screenshots publicly.

9. Limitations and Future Improvements

- MFA enforcement and privileged-access hardening (for example a policy that denies actions without `aws:MultiFactorAuthPresent`).
- Cross-account role access with an external ID or conditions.
- CloudTrail audit logging to a protected S3 bucket.
- Live EC2 role attachment and access validation.
- Automated IAM policy validation and security checks.
- Infrastructure as Code using Terraform, with automated compliance reporting.

10. Cost Management

IAM itself has no charge. Costs can arise from the dedicated S3 bucket (storage and API requests), CloudTrail configuration if enabled, and any running EC2 resources. Review these, empty and delete the bucket, and terminate test instances before leaving the environment active. Check current AWS pricing for the chosen region.

11. Conclusion

The project demonstrates practical IAM configuration and least-privilege design: permissions are granted through groups, the developer policy is scoped to a single bucket without account-wide bucket listing, auditors are limited to read-only access, and EC2 uses a role rather than stored keys. To complete the work, record live permission tests, validate the EC2 role on a running instance, remove or restrict the broadly privileged Admin-Role, and add MFA, CloudTrail and Infrastructure as Code.

Appendix A – Developer Policy (intended final version)

Derived from the policy editor in Figure 4 with the *ListBuckets* (`s3:ListAllMyBuckets`) statement removed, as described in the project notes. Replace the placeholder with the real bucket name.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListProjectBucket",
      "Effect": "Allow",
      "Action": [ "s3:ListBucket" ],
      "Resource": "arn:aws:s3:::iam-security-project-YOURUNIQUEID"
    },
    {
      "Sid": "ManageProjectObjects",
      "Effect": "Allow",
      "Action": [ "s3:GetObject", "s3:PutObject", "s3:DeleteObject" ],
      "Resource": "arn:aws:s3:::iam-security-project-YOURUNIQUEID/*"
    }
  ]
}
```

Appendix B – Glossary

Term	Meaning
IAM	AWS Identity and Access Management – controls who can do what in an AWS account.
User / Group	An identity with credentials / a collection of users that share permissions.
Policy	JSON document that allows or denies actions on resources. Managed (AWS or customer) or inline.
Role	Identity assumed by a service or user that provides temporary credentials.
Instance profile	Container that passes an IAM role to an EC2 instance.
Least privilege	Granting only the permissions required for a task.
ARN	Amazon Resource Name – unique identifier used to scope policies to resources.
MFA	Multi-factor authentication – a second proof of identity at sign-in.
CloudTrail	AWS service that records account activity and API calls for auditing.